

ITO5201 - Essential Reading

Dr. Mario Boley, Department of Data Science and AI, Monash University

The Elements of Machine Learning I

Predictive Modelling

A **predictive model** relates the value t of a **target variable** T to the values $(x_1, \dots, x_d) = \mathbf{x}$ of a number of **input variables** $X_1, \dots, X_d$ through a **prediction function** $y(\mathbf{x}, \mathbf{w})$ that depends on a **parameter** vector $\mathbf{w} = (w_1, \dots, w_M)$. The process of **model fitting** by running a **learning algorithm** is to use a **data** sample $(\mathbf{x}_1, t_1, \dots, \mathbf{x}_N, y_N)$ to determine appropriate parameter values $\mathbf{w}$. In a **regression** problem the target variable T takes on real values and typically has a continuous distribution. In contrast, in a **classification** problem, the target variable takes values from a finite and unordered set of “classes” $\{c_1, \dots, c_K\}$. By “unordered” we mean that c_1 is not “closer” to c_2 than it is to c_3 .

Performance Assessment

A **loss function** $l(t, y)$ measures the quality of a prediction y relative to a target value t . For instance for regression we typically use as loss function the **squared error** $l(t, y) = (t - y)^2/2$. The **generalisation error** of a prediction function $y(\cdot, \mathbf{w})$ is defined as the expected loss of a random data point drawn with respect to the joint distribution of the input and the target variables, i.e.,

$$\mathbb{E}[l(T, y(X, \mathbf{w}))] .$$

The generalisation error is also sometimes called **prediction risk**. Splitting the available data into a **train set** $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_L, y_L)\}$ and a **test set** $\{(\mathbf{x}_{N-L+1}, y_{N-L+1}), \dots, (\mathbf{x}_N, y_N)\}$ gives rise to the train error and test error functions defined as

$$E_{\text{train}}(\mathbf{w}) = \sum_{n=1}^L l(t_n, y(\mathbf{x}_n, \mathbf{w}))$$
$$E_{\text{test}}(\mathbf{w}) = \sum_{n=N-L+1}^N l(t_n, y(\mathbf{x}_n, \mathbf{w}))$$

respectively. These are sometimes referred to as **empirical train/test risk**. If we use only the train set to fit the parameters, typically by minimising the train error,

$$\hat{\mathbf{w}} = \operatorname{argmin}\{E_{\text{train}}(\mathbf{w}) : \mathbf{w} \in \mathbb{R}^M\}$$

then the test error $E_{\text{test}}(\mathbf{w}_L^*)$ is an unbiased estimator of the generalisation error of $\mathbf{w}_L^*$ and a conservative (pessimistic) estimator of the parameter fit obtained from all data, i.e.,

$$\mathbb{E}[E_{\text{test}}(\mathbf{w}_L^*)] = \mathbb{E}[l(T, y(X, \mathbf{w}_L^*))] \geq \mathbb{E}[l(T, y(X, \mathbf{w}_N^*))] .$$

The latter parts holds under the very reasonable assumption that the model performance improves with the training set size. **Cross validation** refers to the process of partitioning the data into K non-overlapping “folds” of roughly equal size.

Model Complexity

Model complexity typically refers to the number or the effective number of learnable parameters of a model. The very related notion of **model flexibility** refers to the capacity of the model to fit collections of systematic or random patterns in the data. **Regularisation** is a technique to control the complexity / flexibility of a model by adding adding a penalty term to the train error as objective function for model fitting, i.e., weights are found to minimise the combination of the train error and a **penalty term**. For instance,

$$\mathbf{w}^* = \operatorname{argmin}\{E_{\text{train}}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2/2 : \mathbf{w} \in \mathbb{R}^M\}$$

where $\lambda \in \mathbb{R}_+$ is referred to as **regularisation parameter**. Larger values for λ make the model less flexible or, put differently, reduce the effective complexity. **Underfitting** can be defined as a situation where the selected model is not flexible enough to capture the systematic patterns in the data. **Overfitting** can be defined as a situation where the model is so flexible that it also captures the random patterns in the data. Typically, overfitting corresponds to a range along a complexity / flexibility axis, e.g., the one defined by the regularisation parameter, where the test error increases with increasing model flexibility.

Model Selection

Model selection in the widest sense refers to choosing

1. the input variables to consider (often chosen from a large set of possible variables),
2. the model class, i.e., in the case of a predictive model, the general form of prediction function (e.g., “polynomial model” or “ K -nearest neighbour model”),
3. **architectural parameters** that determine the concrete set of learnable parameters (e.g., “degree of polynomial M , number of nearest neighbours K ”), and
4. **hyper-parameters** that determine how the model is fitted (e.g., the regularisation parameter).

Importantly, when using a train/test split and selecting a model from a set of options, then the test error is no longer a conservative estimate of the generalisation error of the chosen and fitted model, because the test set was effectively used to “learn” what model to select. To retain an unbiased performance estimate in this situation, we have to perform a three-fold split into a train, a test, and a **validation set**. Instead of a single split into train and validation one can also perform a cross validation for model selection, which is then often referred to as **inner cross validation**. Similarly, one can use several splits into test on the one hand and train and validation on the other resulting in an **outer cross validation**.

The Elements of Machine Learning II

Probabilistic Models

A **probabilistic model** is a conditional distribution of a variable Y given input variable(s) X that additionally depend on a parameter vector $\mathbf{w} \in \mathbb{R}^M$. Depending on the nature of the target variable, a probabilistic model can be given as a probability mass or a probability density function, $p(y|\mathbf{x}, \mathbf{w})$. Note that a probabilistic model is a generalisation of a predictive model, because it naturally also provides a **prediction function**, typically defined as $y(\mathbf{x}, \mathbf{w}) = \mathbb{E}[Y | X = \mathbf{x}, \mathbf{w}]$ in regression and $y(\mathbf{x}, \mathbf{w}) = \operatorname{argmax}_{y \in D_Y} p(y|\mathbf{x}, \mathbf{w})$. Thus, probabilistic models can be thought of as predictive models that also quantify the uncertainty associated with their predictions.

The real target variable T in a supervised learning setting might or might not have a conditional distribution $T|X$ that follows the same distribution as $Y|X$ for some parameter vector $\mathbf{w}$. In the first case, the model is called **well-specified**, and the latter case the model is called **misspecified** in the statistics literature. In any case, it is common to directly write the model using the letter t , i.e., $p(t|\mathbf{x}, \mathbf{w})$.

As previously with predictive models, we aim to fit probabilistic models as closely as possible to a dataset $\{(t_1, \mathbf{x}_1), \dots, (t_N, \mathbf{x}_1)\}$. One way to do that is to maximise the **likelihood function** given by the dataset, i.e., the probability $p(\mathbf{t}|\mathbf{X}, \mathbf{w})$ as a function of the parameters $\mathbf{w}$, where we denote by $\mathbf{t} = (t_1, \dots, t_N)$ the vector of all target values and by $\mathbf{X}$ the input matrix where the i -th row is equal to $\mathbf{x}_i$. A parameter vector $\mathbf{w}_{\text{ml}} \in \mathbb{R}^M$ that maximises the likelihood function is called a **maximum likelihood parameter estimate**. Assuming that the training data points are all independent, we have $p(\mathbf{t}|\mathbf{X}, \mathbf{w}) = \prod_{n=1}^N p(t_n|\mathbf{x}_n, \mathbf{w})$. Moreover, maximising the likelihood function is equivalent to minimising its negative logarithm. Putting these two facts together we see that

$$\mathbf{w}_{\text{ml}} = \operatorname{argmax}_{\mathbf{w} \in \mathbb{R}^M} p(\mathbf{t}|\mathbf{X}, \mathbf{w}) = \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^M} \sum_{n=1}^N -\log p(t_n|\mathbf{x}_n, \mathbf{w}) ,$$

which reveals that maximum likelihood parameter estimation can be understood as test error minimisation where we choose as loss function the negative log likelihood of a single data point.

Uncertainty

In the standard supervised learning setting we predict the value of a target variable T given the value $\mathbf{x}_0$ of an input variable X and past observations of the joint distribution of X and T . However, to act on such a prediction it is crucial to also quantify our **uncertainty** about it, i.e., the degree to which we believe that other outcomes could also be possible. There is a number of sources of uncertainty. Firstly, there is the inherent non-determinism in T given X , called **aleatoric uncertainty**, encapsulated by the conditional distribution $T|X$. Rationally, this uncertainty is irreducible even when observing an arbitrary large amount of training data. In contrast, **epistemic uncertainty** refers to all uncertainties that arise from our *lack of knowledge* about the modelled process, which correspondingly can be reduced when observing an increasing amount of data. This includes **model uncertainty**, i.e., what form does $p_{T|X}(t|\mathbf{x}, \mathbf{w})$ take, as well as **parameter uncertainty**, i.e., what are the true parameters $\mathbf{w}_{\text{true}}$ or, in the case of a misspecified model, the parameters $\mathbf{w}_{\text{opt}}$ that describe the true distribution $T|X$ as closely as possible.

Bootstrap

With a probabilistic model we aim to explicitly represent the aleatoric uncertainty. However, for a finite dataset our estimate of this aleatoric uncertainty may be inflated or deflated. To get a more reliable uncertainty quantification, we have to also represent our epistemic uncertainty. The **bootstrap method** approaches this problem by quantifying the sensitivity of our parameter estimates and other derived quantities (e.g., their train and test error) on the specific dataset we observe. Specifically, the bootstrap method estimates the distribution of any **statistic** (any function of the data) under re-sampling the dataset from the underlying joint distribution of X and T . It does so, similar to cross validation, by repeated sub-sampling of the available data. However, in contrast to cross validation, it uses overlapping samples that are drawn *with* replacement. In detail, a **bootstrap sample** of size $M \leq N$ of the training data $D = (\mathbf{x}_1, t_1), \dots, (\mathbf{x}_N, t_N)$ is a random sub-sample $D' = (\mathbf{x}'_1, t'_1), \dots, (\mathbf{x}'_M, t'_M)$ such that the (b'_n, t'_n) are drawn independently and identically distributed according to the uniform distribution over D . Since we sample with replacement, with overwhelming probability, some elements of the original sample will show up multiple times in the bootstrap sample and other will not show up at all. The latter are sometimes referred to as **out-of-bag** samples, and they can be used to play the role of test data when we use the sub-sampled data points as training data.

Bayesian Models

A **Bayesian model** is an extended probabilistic model where in addition to a conditional distribution $p(y | \mathbf{x}, \mathbf{w})$ we also define a **prior distribution** $p(\mathbf{w})$ over the possible values of the parameters, i.e., we treat the parameters as random variables. This in turn defines a conditional distribution of the parameters given the data

$$p(\mathbf{w} | \mathbf{t}, \mathbf{X}) = \frac{p(\mathbf{t} | \mathbf{w}, \mathbf{X})p(\mathbf{w}, \mathbf{X})}{p(\mathbf{t} | \mathbf{X})}$$

called the **posterior distribution**. The denominator in the above equation, the marginal probability of the targets given the inputs, also called the **marginal likelihood** of the model, is often used as a model selection score when choosing between different Bayesian model variants (as an alternative to cross validation).

To perform predictions with a Bayesian model, one can obtain a single so-called **point estimate** $\mathbf{w}$ of the parameters from the posterior distribution and use the prediction function associated with $p(y | \mathbf{x}, \mathbf{w})$. For this, one typically uses the **maximum a posterior** estimate $\mathbf{w}_{\text{map}} = \text{argmax}_{\mathbf{w}} p(\mathbf{w} | \mathbf{t}, \mathbf{X})$ or the posterior mean $\mathbf{w}_{\text{mean}} = \int_{\mathbb{R}^M} \mathbf{w} p(\mathbf{w} | \mathbf{t}, \mathbf{X}) d\mathbf{w}$. However, instead of using a single conditional target distribution corresponding to a parameter point estimate, one can also average the conditional target distributions for all $\mathbf{w}$ weighted by their posterior probability (density). This forms the Bayesian **predictive distribution**

$$p(y | \mathbf{x}_0) = \int_{\mathbf{w} \in \mathbb{R}^M} p(y | \mathbf{x}_0, \mathbf{w}) p(\mathbf{w} | \mathbf{t}, \mathbf{X}) d\mathbf{w}$$

which takes into account the uncertainty about the parameter estimates provided by the model.